A Novel Center-Boundary Metric Loss to Learn Discriminative Features for Hyperspectral Image Classification

Shaohui Mei, *Senior Member, IEEE*, Zonghao Han, Mingyang Ma, *Student Member, IEEE*, Fulin Xu, and Xingang Li

***Abstract*— Learning discriminative features is crucial for hyperspectral image (HSI) classification. Though metric learning has been applied to learn effective features in HSI classification tasks, existing metric loss functions only consider distance among features of sample pairs but ignore the feature centers and boundaries in the embedding feature space, which limits the discrimination of learned features. In this article, a novel metric loss function named center-boundary metric loss (CBML) is proposed to learn more discriminative features so as to improve HSI classification performance. Unlike the existing metric loss functions, CBML not only considers the distance between sample pairs to enhance intraclass similarity and interclass separability but also pays more attention to the feature centers and boundaries in the embedding feature space that could greatly determine and affect the category of features. Specifically, CBML forces the distance of a sample to its corresponding feature center to be explicitly smaller than that to samples from other classes by a predefined threshold. As a result, the boundaries of different classes will separate an actual distance, which improves the discrimination of learned features. Moreover, in order to improve the training efficiency, a cross mini-batch sampling strategy is further proposed to break through the limitation within the mini-batch by using features between several contiguous mini-batches to sample pairs without increasing the size of the mini-batch. Accordingly, the sampling range of sample pairs is greatly expanded, and the training data is more fully exploited. Experimental results over four benchmark datasets with a typical network for HSI classification demonstrate our proposed method outperforms several state-of-the-arts.**

***Index Terms*— Deep learning (DL), hyperspectral image (HSI) classification, metric learning, metric loss functions.**

I. Introduction

HYPERSPECTRAL images (HSIs) have hundreds of spectral bands ranging from visible to near-infrared, containing abundant spectral and spatial information. On this basis, HSI classification aims to accurately characterize and identify the category of materials of our interest, which is widely employed in various applications, such as land-cover mapping [1], land change monitoring [2], and anomaly detection [3], [4].

In recent years, with the rapid development of the deep learning (DL) technique, DL-based methods have been successfully applied to HSI classification, such as stacked autoencoders (SAEs), deep belief networks (DBNs), recurrent neural networks (RNNs), and convolutional neural networks (CNNs) [5], [6], [7], [8], [9]. In the above-mentioned networks, the CNN architecture captures local information by using local receptive fields, shared parameters, and spatial subsampling, while being able to skillfully combine spectral features with spatial context information in HSI data, which makes it become a research hotspot in HSI classification. For instance, Makantasis et al. [10] proposed a model that automatically captures high-level spatial features by using CNN and multilayer perceptron (MLP). Mei et al. [7] designed a peer feature learning CNN based on supervised and unsupervised models to learn sensor-specific spatial–spectral features. Zhang et al. [9] designed a cross-domain framework for graph information aggregation based on a mechanism combining data analysis and few-shot learning, which can solve problems such as differences in spatial and spectral resolutions of different sensors and differences in the same land cover category.

In recent years, transformer [11], [12], [13], [14], [15] has also been applied to HSI classification. For example, He et al. [11] cascaded CNNs with transformers, in which CNNs are used to extract spatial features, while transformers are more focused on extracting spectral features. Tan et al. [12] fed morphological profile (EMP) features of the HSI into a transformer to extract patch-level and pixel-level features, respectively, which can make full use of the global and local information of EMP. He et al. [13] introduced an HSI transformer classification network to capture subtle spectral differences and deliver local spatial context information by embedding convolution operations in the transformer structure. Yang et al. [14] proposed a BERT-based HSI classification framework, which can capture global context and also focus on important locations with less redundancy.

However, neither CNN-based nor transformer-based methods can effectively exclude the influence of different pixels and various noises in feature extraction. Therefore, to enhance the

Manuscript received 20 November 2023; revised 3 January 2024; accepted 29 January 2024. Date of publication 5 February 2024; date of current version 15 February 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62171381 and Grant 62201445 and in part by the Zhejiang Province "Pioneering Soldier" and "Leading Goose" Research and Development Project under Grant 2023C01027. *(Corresponding author: Shaohui Mei.)*

Shaohui Mei, Zonghao Han, Fulin Xu, and Xingang Li are with the School of Electronics and Information, Northwestern Polytechnical University, Xi'an 710129, China (e-mail: meish@nwpu.edu.cn).

Mingyang Ma is with the School of Information and Communications Engineering, Xi'an Jiaotong University, Xi'an, Shaanxi 710049, China.

Digital Object Identifier 10.1109/TGRS.2024.3362391

1558-0644 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:37:13 UTC from IEEE Xplore. Restrictions apply.

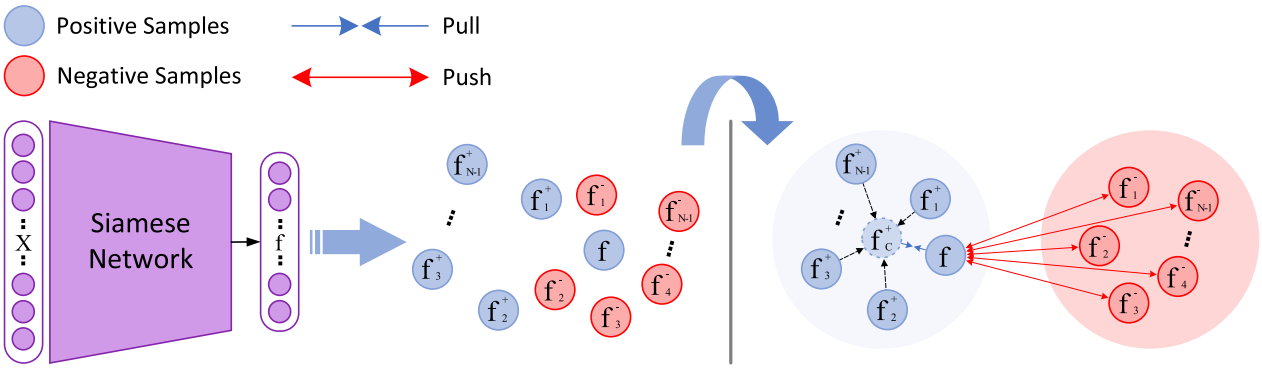


Fig. 1. (Left) Illustration of our proposed CBML. Embedding vectors f are mapped from training samples through a Siamese network and they are not separated from each other in the general embedding space. (Right) CBML pulls the feature vector of the sample to their corresponding feature center while pushing it away from other feature vectors of other classes. Therefore, the features of different classes will explicitly have definite boundaries in the embedding space.

discrimination of features learned by deep neural networks, metric learning [16], [17], [18], [19], [20], [21] has been applied to this filed.

Metric learning aims to learn a mapping from original data points to a low-dimensional dense vector space so that the embedded vectors of similar samples should be closer while dissimilar ones are pushed apart from each other. Due to this advantage, metric learning has been applied to various tasks, such as image retrieval [22], [23], [24], [25], image classification [26], [27], [28], [29], [30], face recognition [31], [32], [33], few-shot learning [34], [35], [36], object tracking [37], [38], [39], and person reidentification [40], [41], [42], [43].

Recently, various metric loss functions are designed for deep CNNs to enhance their feature representation discriminative ability by imposing distance-based metrics (such as Euclidean distance, Markov distance, and cosine distance) [44], [45], [46], [47], [48], [49], [50] in the embedding space under a Siamese [51], [52] or triplet network [53], [54]. For example, contrastive loss [44] attempts to force the distance between positive pairs to be smaller than a predefined threshold while the distance between negative pairs is larger than another predefined threshold. Triplet loss [45] aims to force anchor-positive distance smaller than the anchor-negative distance by a predefined margin using a triplet input composed of an anchor sample, a positive and a negative sample. Other similar metric loss functions have also been proposed by considering the distance among more pairs of samples from different classes. Quadruplet loss [55], [56], [57] constraints the distance of sample pairs from three different classes. Multiclass N-pair loss [48] optimizes the distance between $N-1$ negative samples and one positive sample.

Existing works [58], [59] mainly consider the distance among features of training sample pairs to enhance intraclass similarity and interclass separability. An intuition is that one class may be determined by its feature center and boundary in the embedding feature space, which has not been considered in existing metric learning approaches. Therefore, by considering class feature centers and boundaries in the embedded feature space, a novel metric loss function named center-boundary metric loss (CBML) is proposed in this article to learn more discriminative features. CBML attempts to force the distance of a training sample to its corresponding feature center to be definitely smaller than that of samples from other classes, where the distance comparison is constrained by a predefined threshold. As a consequence, the boundaries of different classes will explicitly have a definite distance in the embedding feature space, as illustrated in Fig. 1.

For the purpose of computing the distance of a sample to its corresponding feature center, CBML adopts a linear neural network without bias to directly learn it instead of using geometric centers or parametric centers [31]. By this means, it can effectively alleviate computational overhead and make the training process more stable. Moreover, the feature centers can also be learned during the training process. Hence, the class of samples can be directly determined by the distance to class feature centers instead of training an additional classifier such as K-NN and SVM, as shown in Fig. 2.

Generally, deep CNNs are trained with mini-batch stochastic gradient descent methods that allow a mini-batch of data at each iteration in the training process [60], [61], which can be directly applied to metric learning [62], [63]. Nevertheless, when sample pairs are selected for training, the selection of inputs will be limited within a small size of mini-batch, resulting in training samples being incapable of being completely utilized. In order to fully explore informative information in training samples, a training strategy called memory bank was applied [64] to metric learning, which significantly expands the sampling range of training data by utilizing the sample features from the previous iteration in the current training iteration. However, the features of the same sample in different iterations differ greatly at the beginning of the training process, which may lead to nonconvergence. In our preliminary experiments, we found a "progressive feature similarity," meaning that the feature of the same sample will become more and more similar as the training continues. According to such "progressive feature similarity" phenomenon, an improved memory bank-based training strategy called cross mini-batch sampling strategy is proposed by constructing a queue to store the features of samples from several contiguous mini-batch and combine them with the features of the current mini-batch to expand the sampling range of sample pairs at the latter part of the training process.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:37:13 UTC from IEEE Xplore. Restrictions apply.

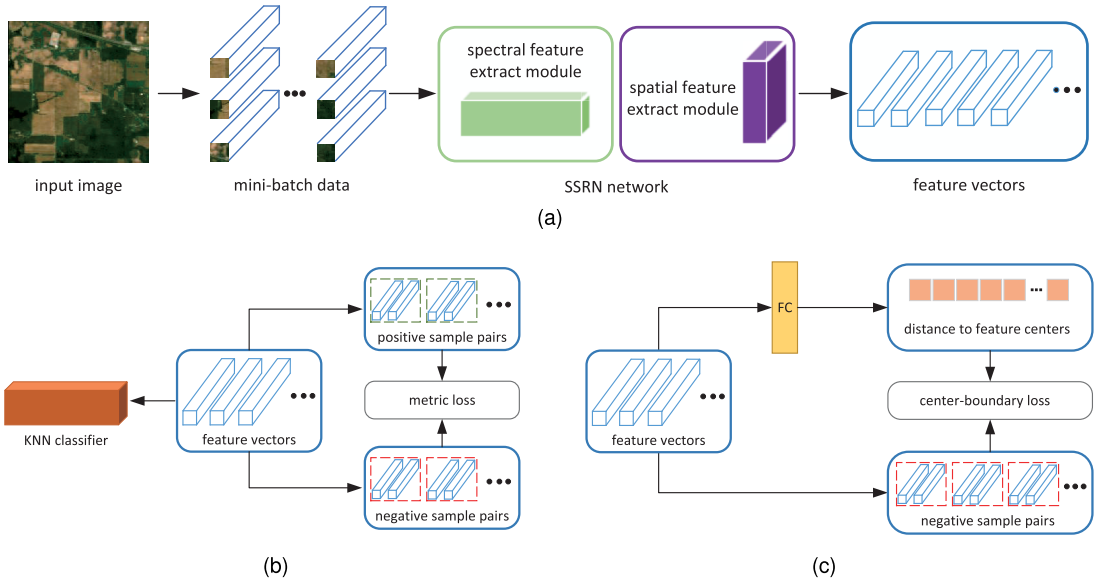


Fig. 2. Illustrations of metric loss functions and CBML training procedures. (a) With the size of $7 \times 7 \times 200$ sampled cubes from the whole data are used to form mini-batches data as inputs and then spatial–spectral feature vectors are obtained through the SSRN network. (b) Illustration of metric loss function training process. (c) Illustration of CBML training process.

In summary, our main contributions are as follows.

1) A novel metric loss function named CBML, is designed to learn more discriminative features for HSI classification by driving the distance of a training sample to its feature center is smaller than that to samples from other classes. To learn the distance of a sample to its corresponding feature center, a linear neural network without bias is adopted. In this way, the class of samples can be directly determined by the distance to class feature centers instead of training an additional classifier like other existing metric loss functions.
2) An improved memory bank-based sampling strategy named cross mini-batch sampling is proposed according to the "progressive feature similarity" phenomenon that features of the same samples will become more and more similar as the training goes on. The proposed sampling strategy significantly expands the sampling range of sample pairs while not extending the size of mini-batches. By this strategy, the information in training data is allowed to be fully exploited so as to improve the training efficiency.

The remaining of the article is organized as follows. Section II presents the related work of various metric loss functions including contrastive loss, triplet loss, and multisimilarity loss. Section III introduces the details of our proposed CBML and cross-mini-batch sampling strategy. The experiments and analysis of HSI classification results over four benchmark datasets are reported in Section IV. Finally, the conclusion is drawn in Section V.

II. Related Work

In metric learning, training data is usually composed of sample pairs, in which positive sample pairs refer to samples with the same class label, and negative sample pairs are consisted of samples with different class labels. After that, the feature vectors of sample pairs are extracted by the siamese structure of deep CNNs. Therefore, metric loss functions have access to effectively constrain the distance between feature vectors of sample pairs and make the distance between positive pairs as close as possible while negative pairs are far away from each other. In this section, we introduce three well-known metric loss functions, including contrastive loss, triplet loss, and multisimilarity loss.

A. Metric Loss Functions

1) Contrastive Loss: Contrastive loss [44] is a typical metric loss function that aims to make the distance (d_p) between positive pairs smaller than some threshold (m_p), and the distance (d_n) between negative pairs larger than some threshold (m_n)

$$L_{\text{contrastive}} = \left[d_p - m_p\right]_+ + \left[m_n - d_n\right]_+ \tag{1}$$

where (m_p) is set to 0 in many implementations, and $[*]_+$ means the maximum value between 0 and $*$. The disadvantage of contrastive loss is that the same distance threshold is applied to all the sample pairs even though there may be a large variance among them.

2) Triplet Loss: The triplet loss [20] theoretically addresses the issue of the same threshold for all pairs by constructing a triplet which consists of an anchor, a positive and a negative sample, and makes the distance $\left(d_{ap}\right)$ between anchor and positive sample is smaller than the distance (d_{an}) between anchor and negative sample by a predefined margin (m)

$$L_{\text{triplet}} = \left[d_{ap} - d_{an} + m\right]_+. \tag{2}$$

The triplet loss explicitly places fewer restrictions on the embedding feature space and allows the model to account for interclass dissimilarities. Nevertheless, selecting the triplets is a grand challenge. Generating all the possible triples before training may result in many triplets easily satisfied (2), which

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:37:13 UTC from IEEE Xplore. Restrictions apply.

would not contribute to training models and bring about slower convergence because they would still be transited through the network. Therefore, it is crucial to select hard triplets that are effective and make contributions to improve the model.

To address this challenge, Schroff et al. [45] proposed a hard triplet mining strategy that focuses on the online generation and selects the hard positive/negative exemplars from within a mini-batch data. In this method, for each sample x^a, they select a positive sample x^p satisfies $\text{argmax}_{x^p} \| f(x^p) - f(x^a) \|_2^2$ and a negative sample x^n satisfies $\text{argmin}_{x^n} \| f(x^n) - f(x^a) \|_2^2$, where $f(\cdot)$ is the embedding feature vector. As a result, the performance of the network and convergence speed are improved effectively. The only drawback in this hard sample pair mining method is that the training data is not fully utilized because only the hardest sample exemplars are selected, but other sample pairs that also can make contributions to train the model are ignored.

3) Multisimilarity Loss: Multisimilarity loss [46] has found the flaw of triplet loss and improved it by two steps: pair mining and weighting. The purpose of pair mining is to select informative sample pairs by measuring the relative similarity (as opposed to relative distance) between positive and negative pairs that share the same anchor. In brief, a negative pair is sampled if the similarity is greater than the hardest positive pair (with the lowest similarity), while a positive pair is selected when the similarity is less than the negative one which has the largest similarity. Consequently, the sampling range of sample pairs is effectively improved compared with selecting the hardest positive/negative exemplars. After that, the second step attempts to further weigh the selected pairs more accurately by considering both positive relative similarity and negative relative similarity. The weighting scheme is encouraged by binomial deviance loss (considering positive relative similarity) [40] and lifted structure loss (using negative relative similarity) [46]. Finally, multisimilarity loss integrates pair mining and weighting mechanisms into a single framework and can be formulated as

$$L_{\text{multisimilarity}} = \frac{1}{m} \sum_{i=1}^{m} \left\{ \frac{1}{\alpha} \log \left[1 + \sum_{k \in \mathcal{P}_i} e^{-\alpha(S_{ik} - \lambda)} \right] + \frac{1}{\beta} \log \left[1 + \sum_{k \in \mathcal{N}_i} e^{\beta(S_{ik} - \lambda)} \right] \right\} \quad (3)$$

where S_{ik} means the similarity between sample pairs, $\mathcal{P}_i$ and $\mathcal{N}_i$ are collections of positive/negative sample pairs, and α, β, λ are hyperparameters.

III. Proposed Method

In this section, a new metric loss function named CBML is designed to significantly enhance the discrimination of sample features in the embedding feature space. This new metric loss pulls the feature vector of the sample to its corresponding feature center while pushing it away from other feature vectors of different classes. Thus, the boundaries of different classes can be constrained to have a definite distance in the embedding feature space. In order to further make full use of informative information in training data, we propose an improved training strategy based on the memory bank method named cross mini-batch sampling strategy, which is capable of definitely expanding the sampling range of sample pairs.

A. Center-Boundary Metric Loss

Suppose that, given a set of training samples $\{(x_i, y_i), i = 1, \ldots, n\}$ of $K(0, 1, 2, \ldots, K-1)$ classes, where x_i means the training data and y_i is its corresponding label. Let $f(x_i)$ be an embedding feature vector of a sample, where $f : \mathbb{R}^N \longrightarrow \mathbb{R}^D$ is a deep CNN. In our approach, for a given sample, the positive pair consists of it and its corresponding feature center. In contrast, the negative pairs are composed of it and other samples from different classes. The goal of CBML is to force the distance of a training sample to its counterpart feature center to be definitely smaller than that of samples from different classes by a predefined margin. As a consequence, the boundaries of different classes will explicitly interval a definite distance in the embedding feature space. The CBML can be formed as follows:

$$L_{\text{CBML}} = \left[d_{ic} - d_{ij} + m \right]_+ \quad (4)$$

where d_{ic} represents the distance of a sample to its corresponding feature center, while d_{ij} is the distance to other negative samples.

For a negative sample pair $\{x_i, x_j\}$, the relative distance d_{ij} can be computed directly by a distance metric such as dot product distance

$$d_{ij} = -f(x_i) \cdot f(x_j). \quad (5)$$

However, for positive sample pairs, it is hard to calculate the distance between them because the feature centers are exactly unavailable.

In general, the feature center of samples can be expressed as the geometric center of the embedding feature vectors

$$C_k = \frac{1}{n} \sum_{y_i = k} f(x_i) \quad (6)$$

where C_k is the feature centers for the samples of kth classes. However, the geometric centers are incapable of accurately representing the feature centers due to variances among samples of different classes, and it is inefficient and even impractical to average the features of every class in each iteration during the training process.

To address this problem, center loss [31] randomly initializes the feature centers and updates them within a mini-batch as the training goes on. Similar to center loss, triplet center loss [65] also uses parametric centers instead of geometric centers and combines center loss with triplet loss to learn more discriminative features. Nevertheless, such parametric centers are updated at each iteration based on a mini-batch which is extremely unstable. Thus, it has to be under the joint supervision of softmax loss in training time. For this reason, such a scheme dramatically increases the computational overhead. Inspired by [35], which applies a neural network to learn the variance between query and support images, we come up with a completely different approach. Instead of finding the feature centers and then computing the distance of a sample to its

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:37:13 UTC from IEEE Xplore. Restrictions apply.

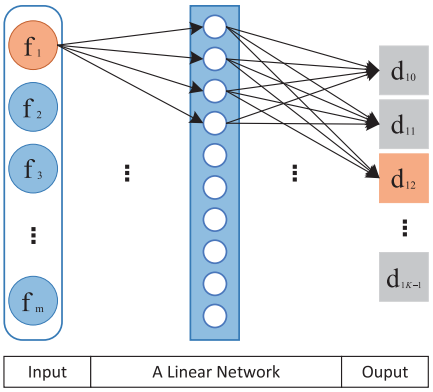


Fig. 3. Illustration of our method to compute the distance of a sample to its corresponding feature center. Taking the sample x_1 form a mini-batch data $\{(x_1, y_1), (x_2, y_2), \ldots, (x_m, y_m)\}$ as an example, and its class label y_1 is 2. Feature vector $f(x_1)$ (as shown in the figure is f_1) is the embedding of it. Afterward, a linear network uses the feature vector $f(x_1)$ as input and outputs the distance $(d_{10}, d_{11}, \ldots, d_{1K-1})$ of the sample to all the feature centers. Then, the distance of this sample to its corresponding feature center (d_{12}) can be obtained in line with its class label.

feature center under a definite distance metric, we use a fully connected network without bias to directly learn the distance between samples and their corresponding feature centers. The input of such a linear network is the embedding feature vector of a training sample, and the output is the distance of it to the K feature centers. Then, the distance of the training sample to its counterpart feature center can be acquired according to its class label. Fig. 3 shows an example to illustrate our method. Formally, the distance of a sample to its feature center can be formulated as follows:

$$d_{ic} = (W^T f(x_i))_k \tag{7}$$

where $W \in \mathbb{R}^{D\times K}$ is the weight matrix of the linear neural network and the subscript k is the corresponding class label of the sample x_i. In comparison to parametric centers, our proposed method can adaptively assign weights according to the variance among samples, thus making the training process more stable and alleviating the effects of the mini-batch. More importantly, training a linear neural network has much less computation than updating the parametric centers.

In order to further improve the training efficiency, the contribution of different sample pairs should be treated unequally during the training process. To achieve this, we introduce the pair mining and weighting method advocated by Wang et al. [46] in CBML, and then the gradient weights of training pairs are weighted according to the distance between sample pairs in the embedding feature space. As a result, the sample pairs that are key to feature separability are emphasized to make full use of informative information in all sample pairs and speed up convergence. Similar to multisimilarity loss, the first step of pair mining is to find all the informative sample pairs in the training data. Specifically, a negative pair is compared to the hardest positive pair, while a positive pair is composed of a sample and its corresponding feature center. Formally, given a sample x_i, a negative pair $\{x_i, x_j\}$ is selected if d_{ij} satisfies the condition

$$d_{ij} < \max_{y_j \neq c_k} d_{ic} + \lambda \tag{8}$$

where λ is a predefined margin.

In the second step, we adopt a soft weighting scheme that uses LogSumExp (LSE) and SoftPlus (SP) functions to weight the sampled pairs precisely like the multisimilarity loss but less complicated. Given a vector $\{x_1, x_2, x_3, \ldots, x_n\}$, the LSE function of it is defined as

$$\mathrm{LSE}(\{x_1, x_2, x_3, \ldots, x_n\}, \alpha) = \frac{1}{\alpha}\sum_{i=1}^{n} \log x_i. \tag{9}$$

Given a scalar x, the SP function is formulated as

$$\mathrm{SP}(x, \alpha) = \frac{1}{\alpha}\log(1 + e^{\alpha x}). \tag{10}$$

Finally, we integrate pair mining and weighting scheme into a single framework, and our proposed CBML is formulated as

$$\begin{aligned} L_{\mathrm{CBML}} &= \mathrm{SP}\left\{\mathrm{LSE}\left[\sum_{i\in\mathcal{M}}\sum_{j\in\mathcal{N}_i}\left(d_{ic} - d_{ij} + \lambda\right)\right]\right\} \\ &= \frac{1}{\alpha}\log\left\{1 + \sum_{i\in\mathcal{M}} e^{\alpha(d_{ic})}\cdot\sum_{j\in\mathcal{N}_i} e^{\alpha\left(\lambda - d_{ij}\right)}\right\} \end{aligned} \tag{11}$$

where for a given sample x_i, $\mathcal{N}_i$ is the index set of its selected negative pairs, $\mathcal{M}$ means a mini-batch data, and α and λ are hyper-parameters to train network.

As indicated in (11), the gradient weights of a positive pair can be computed as

$$\begin{aligned} \mathcal{W}_{ic}^{+} &= -\frac{\partial L_{\mathrm{CBML}}}{\partial d_{ic}} \\ &= -\frac{\sum_{j\in\mathcal{N}_i} e^{\alpha\left(\lambda - d_{ij}\right)}\cdot e^{\alpha(d_{ic})}}{1 + \sum_{i\in\mathcal{M}} e^{\alpha(d_{ic})}\cdot\sum_{j\in\mathcal{N}_i} e^{\alpha\left(\lambda - d_{ij}\right)}} \\ &= \left(e^{(-\alpha L_{\mathrm{CBML}})} - 1\right)\cdot\frac{e^{\alpha(d_{ic})}}{\sum_{i\in\mathcal{M}} e^{\alpha(d_{ic})}} \end{aligned} \tag{12}$$

and the gradient weights of a negative pair are

$$\begin{aligned} \mathcal{W}_{ij}^{-} &= -\frac{\partial L_{\mathrm{CBML}}}{\partial d_{ij}} \\ &= \frac{e^{\alpha\left(\lambda - d_{ij}\right)}\cdot\sum_{i\in\mathcal{M}} e^{\alpha(d_{ic})}}{1 + \sum_{i\in\mathcal{M}} e^{\alpha(d_{ic})}\cdot\sum_{j\in\mathcal{N}_i} e^{\alpha\left(\lambda - d_{ij}\right)}} \\ &= \left(1 - e^{(-\alpha L_{\mathrm{CBML}})}\right)\cdot\frac{e^{\alpha\left(\lambda - d_{ij}\right)}}{\sum_{j\in\mathcal{N}_i} e^{\alpha\left(\lambda - d_{ij}\right)}}. \end{aligned} \tag{13}$$

Analyzing (12) and (13), we can find that the gradient weights of pairs update dramatically according to the distance between positive and negative sample pairs. Therefore, the contributions of different sample pairs to train networks are treated unequally during the training processing, and feature separability is emphasized by making full use of informative information in sample pairs.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:37:13 UTC from IEEE Xplore. Restrictions apply.

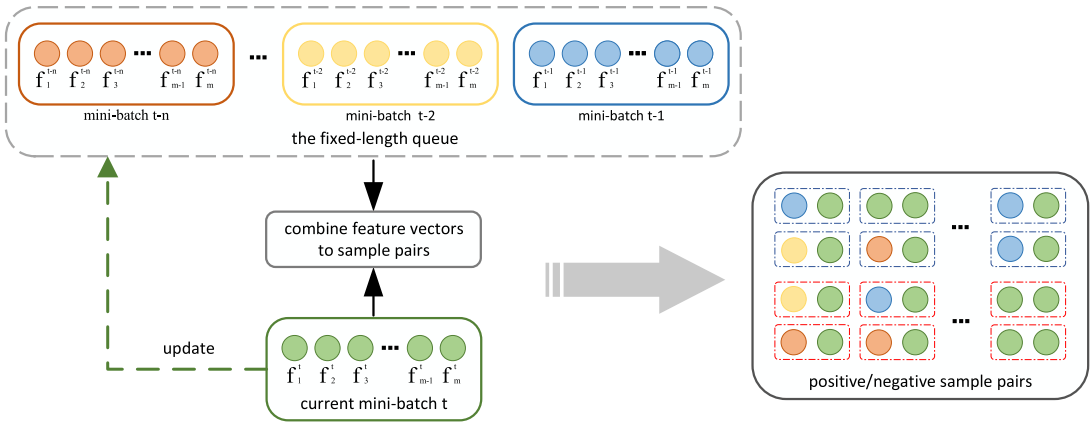


Fig. 4. Illustration of cross mini-batch sampling strategy. $\{f_1^t, f_2^t, f_3^t, \ldots, f_m^t\}$ are feature vectors of the current mini-batch t. The queue (gray rounded dotted line box) saves feature vectors of n consecutive mini-batch from mini-batch $t-n$ to mini-batch $t-1$. Then, our proposed cross mini-batch sampling strategy combines them to select positive (red dotted box) and negative pairs (blue dotted box). In this way, the range of sample pairs is lifted $O(m^2)$ to $O((m+n\times m)^2)$ which significantly expands the range of sample pairs. For the next mini-batch, the feature vectors of mini-batch $t-n$ are replaced by mini-batch t to update the queue.

B. Cross-Mini-Batch Sampling Strategy

Generally, metric learning approaches use gradient descent-based optimization methods to train deep CNNs during the training process. These approaches first randomly sample pairs or triplets to construct training batches and compute loss on the individual pairs or triplets within the mini-batch. However, the sampling is limited to a mini-batch of training samples, which does not make full use of information in the whole training data. In addition, if the samples of a certain class are missed in the current mini-batch, the direction of gradient descent would be altered, resulting in model nonconvergence.

A straightforward strategy to address the issue is to expand the size of the mini-batch. However, it is impractical to greatly expand the size of the mini-batch due to the limitation of the GPU memory. Actually, during the training process, the metric loss is computed by the feature vectors of samples in the embedding space. Consequently, a memory bank method [64], which extends the range of feature vectors instead of expanding the size of the mini-batch, is an effective way. The memory bank-based training strategy constructs a feature vectors dictionary to save the feature vectors of training samples, and the size of the dictionary is much larger than a mini-batch, even the whole training data. During the training, the feature vectors of the samples are stored in the dictionary in the current iteration, and the previous feature vectors of the samples are replaced by the latest ones in the next iteration. By this method, the feature vectors in a mini-batch are combined with the feature vectors stored in the dictionary to sample more pairs, greatly expanding the sampling range of pairs and improving the utilization of training data. Nevertheless, this method has a drawback due to the phenomenon of "feature obsolescence," which refers to the fact that the features extracted from the model between batches are not guaranteed to be identical due to the back-propagation algorithm to update the weights of the model's parameters within the batch during the training process. This phenomenon can lead to the model becoming unstable, or even failing to converge, when using the memory bank-based training strategy.

In preliminary experiments, we also find an interesting phenomenon that the error of the feature vectors of the sample in the adjacent serval epochs becomes smaller and finally remains unchanged with the progress of training. We called this phenomenon as "progressive feature similarity." Furthermore, on this basis, we propose an improved memory bank-based training strategy named cross mini-batch sampling to alleviate "feature obsolescence" phenomenon. This method constructs a fixed-length queue to store the feature vectors of training samples in adjacent n mini-batches when the phenomenon of "progressive feature similarity" appeals and the feature vectors in the queue are continuously updated by the latest ones as the training process goes on. Similar to the memory bank-based strategy, we also combine the feature vectors of the current mini-batch with the feature vectors in the queue to expand the sampling range of sample pairs, as illustrated in Fig. 4. Supposing the size of a mini-batch data is m, and then the size of the queue is $(n\times m)$, so the sampling range of pairs is lifted from $O\left(m^2\right)$ to $O\left((m+n\times m)^2\right)$, which significantly extends the range of sample pairs and thus makes full use of training samples. A step-by-step algorithm of our proposed cross mini-batch sampling strategy is shown in Algorithm 1.

C. Process of Our Method

In our method, we adopt the SSRN model advocated by [66] as our base network to extract features from HSI data. In this network, the spectral and spatial residual blocks consecutively learn robustness features from abundant spectral signatures and spatial contexts in HSI data. The overall structure of SSRN can be seen in Fig. 5. Then, we employ our proposed CBML to facilitate model training to learn more discriminative features by enforcing the distance of a sample to its corresponding feature center to be explicitly smaller than that to samples from other classes by a predefined threshold. The proposed cross mini-batch sampling strategy is not used at the early stage in the training process to prevent unstable model training or nonconvergence caused by the "feature obsolescence." When "progressive feature similarity" appears, our sampling strategy

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:37:13 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Cross Mini-Batch Sampling Strategy

Input: $\mathbb{D} = \{(x^n, y^n)\}_{n=1}^{N}$
Output: Neural network parameter θ
Repeat:
Randomly shuffle the samples in training set $\mathbb{D}$
for $n = 1 \ldots N$ **do**
Extract the samples (x^n, y^n) from $\mathbb{D}$
Calculate the sample features $\boldsymbol{F} = \{\boldsymbol{f}_1, \cdots, \boldsymbol{f}_M\}$
Sample positive and negative pairs from $\{\mathbb{Q}, \boldsymbol{F}\}$
Calculate the loss and back propagation
Put $\boldsymbol{F}$ in $\mathbb{Q}$
if $\mathbb{Q}$ is full **then**
$\mathbb{Q}$ out of the oldest features
end if
end for
until Reach preset training time

TABLE I
DETAILS OF LAND COVER CLASSES IN IP

Class	Name	Number
1	Alfalfa	46
2	Corn-notill	1428
3	Corn-mintill	830
4	Corn	237
5	Grass-pasture	483
6	Grass-trees	730
7	Grass-pasture-mowed	28
8	Hay-windrowed	478
9	Oats	20
10	Soybean-notill	975
11	Soybean-mintill	2455
12	Soybean-clean	593
13	Wheat	205
14	Woods	1265
15	Buildings-Grass-Trees-Drives	386
16	Stone-Steel-Towers	93

is employed to expand the sampling range of samples to fully make use of informative information in the training data, thus improving the model performance. In the testing time, the class of a sample can be directly determined by its distance to class feature centers instead of training an additional classifier like other metric losses.

IV. EXPERIMENT

A. Datasets

In order to demonstrate the performance of CBML and cross mini-batch sampling strategy, we conduct extensive experiments on four benchmark HSI datasets, including Indian Pines (IP), Pavia Centre (PC), Pavia University (PU), and Salinas (SA).

1) The IP dataset was imaged by the Airborne Visible Infrared Imaging Spectrometer (AVIRIS) in 1992 of an Indian pine tree in Indiana, USA. It has 145×145 pixels with 200 wavebands. The dataset includes 16 classes (can be seen in Table I). In our experiments, only 13 categories with abundant labeled samples are selected by abandoning three classes (including alfalfa, grass-pasture-mowed, and oats) with few samples for training.

TABLE II
DETAILS OF LAND COVER CLASSES IN PC

Class	Name	Number
1	Water	824
2	Trees	820
3	Asphalt	816
4	Self-Blocking Bricks	808
5	Bitumen	808
6	Tiles	1260
7	Shadows	476
8	Meadows	824
9	Bare Soil	820

TABLE III
DETAILS OF LAND COVER CLASSES IN PU

Class	Name	Number
1	Asphalt	6631
2	Meadows	18649
3	Gravel	2099
4	Trees	3064
5	Painted metal sheets	1345
6	Bare Soil	5029
7	Bitumen	1330
8	Self-Blocking Bricks	3682
9	Shadows	947

2) The PC dataset was captured by a Reflective Optics System Imaging Spectrometer (ROSIS) in Pavia, northern Italy, which contains nine classes (as described in Table II) and has 1096×715 pixels with 102 wavebands. The geometric resolution of it is 1.3 m.
3) The PU dataset was also gathered by ROSIS over the city of Pavia, Italy. It consists of 610×340 pixels with 103 spectral bands ranging from 0.38 to 0.86 μm. The dataset is divided into nine classes (as shown in Table III).
4) The SA dataset was obtained by the Airborne Visible/Infrared Imaging Spectrometer (AVIRIS), which was taken in California, USA, in 1998. It has 512×217 pixels with a spatial resolution is 3.7 m for every pixel, 204 useful spectral wavebands, and 16 kinds of Land Covers (as shown in Table IV).

B. Evaluation Metrics

To measure the performance of classification effectively, we select three widely used quantitative indicators in our experiments: the overall accuracy (OA), the average accuracy (AA), and the Kappa coefficient (KA). The OA represents the overall percentage of correctly classified pixels for the whole test pixels. The AA can be computed by the average classification of all the land cover classes. The KA is an index to measure the consistency level between classification performance and the counterpart ground truth.

C. Settings

To train the model, we use the Adam optimizer with a ReduceLROnPlateau strategy which decays the learning rate by a factor of 0.1 when the loss has stopped improving and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:37:13 UTC from IEEE Xplore. Restrictions apply.

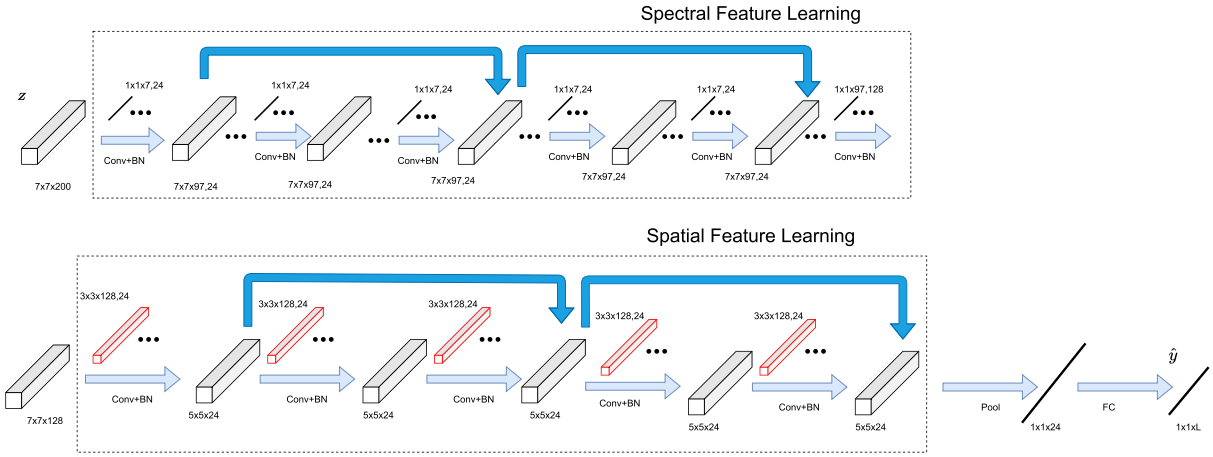


Fig. 5. Overall structure of SSRN.

TABLE IV
DETAILS OF LAND COVER CLASSES IN SA

Class	Name	Number
1	Brocoli-green-weeds-1	2009
2	Brocoli-green-weeds-2	3726
3	Fallow	1976
4	Fallow-rough-plow	1394
5	Fallow-smooth	2678
6	Stubble	3959
7	Celery	3579
8	Grapes-untrained	11271
9	Soil-vinyard-develop	6203
10	Corn-senesced-green-weeds	3278
11	Lettuce-romaine-4wk	1068
12	Lettuce-romaine-5wk	1927
13	Lettuce-romaine-6wk	916
14	Lettuce-romaine-7wk	1070
15	Vinyard-untrained	7268
16	Vinyard-vertical-trellis	1807

the initial learning rate is 0.001. The size of the mini-batch is set to 32 in the training time. We set the maximum number of epochs to train the model as 500 and divide 10% of the training set into the validation data at the beginning of each epoch. After the training of the current epoch, we test the performance of the model on the validation data. When reaching the maximum number of epochs, we select the model with the best results on the validation data for testing. All the experiments are implemented on Ubuntu 18.04 and a GPU of NVIDIA GeForce RTX2080Ti with 12 GB memory. We repeated all the experiments ten times and averaged the evaluation metrics.

D. Comparison With State-of-the-Art Methods

To prove the superiority of our proposed method, several state-of-the-art methods are adopted for comparision, including M3D-DCNN [67], HybridSN [68], DFFN [69], RSSAN [70], AB-LSTM [71], SpectralFormer [73], and GAHT [72]. M3D-DCNN proposed a multiscale 3-D deep CNN that jointly learns both 2-D multiscale spatial features and 1-D spectral features from HSI data. HybridSN is a 3-D CNN that combines spectral and spatial information, followed by a 2-D CNN that focuses on spatial processing. DFFN employs residual learning to increase the depth of the network and merges the outputs of different hierarchical layers to further improve the classification performance. RSSAN is an end-to-end residual spectral–spatial attention network that aims to mine spectral–spatial information from raw 3-D HSI cubes. AB-LSTM proposed an attention-based bidirectional long–short-term memory network, which aims to emphasize the effective information and reduce the redundancy among the spatial–spectral context of pixels. SpectralFormer is a transformer-based backbone network that is capable of learning spectrally local sequence information from adjacent bands of the HSI. GAHT is also a transformer-based network that employs a new grouped pixel embedding module to confine multihead self-attention to the local spatial–spectral context. These baseline methods employ the widely used cross-entropy loss to train the model; 10, 20, and 30 cubes with the spatial size of 7×7 are randomly sampled from four HSI datasets as training samples. In the training time, the length of the queue is set to 32.

Fig. 6 displays the classification results with OA of different methods on IP, PU, SA, and PC datasets. As shown in the figure, our proposed method is significantly better than other baseline approaches, especially in the limited training samples. For instance, our approach achieves OA of 79.74% and 92.74% on IP and PU datasets, using only ten training samples, outperforming other methods by a large margin. The reason for this result is that the traditional classification loss is prone to overfitting under insufficient training data, resulting in less robustness of the features learned by the deep neural networks. Conversely, our CBML employs the metric learning mechanism to impose distance constraints between samples and their corresponding feature centers, which allows the model to learn more discriminative features, significantly improving classification results despite limited training samples. In addition, as the number of training samples increases, the performance of our methods further improves.

Tables V–VIII show the classification results of different methods on IP, PU, SA, and PC datasets with 20 training

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:37:13 UTC from IEEE Xplore. Restrictions apply.

TABLE V
CLASSIFICATION RESULTS OF DIFFERENT METHODS ON IP DATASET WITH 20 TRAINING SAMPLES

Class	M3D-DCNN [67]	HybridSN [68]	DFFN [69]	RSSAN [70]	AB-LSTM [71]	SpectralFormer [73]	GAHT [72]	Proposed
2	44.84 ± 8.35	56.80 ± 9.68	52.19 ± 13.40	46.46 ± 5.89	26.95 ± 34.19	38.96 ± 7.46	71.36 ±3.97	**91.38 ± 1.34**
3	55.73 ± 8.52	66.84 ± 7.45	70.69 ± 5.21	63.28 ± 26.44	16.25 ± 22.24	49.78 ± 3.29	73.83 ± 7.33	**98.16 ± 0.43**
4	90.51 ± 5.96	90.23 ± 11.76	97.60 ± 2.23	77.79 ± 18.13	22.03 ± 26.71	70.60 ± 5.64	99.26 ± 0.75	**100.00 ± 0.00**
5	86.61 ± 2.46	95.40 ± 0.55	87.21 ± 6.13	66.26 ± 32.78	46.91 ± 33.70	66.57 ± 5.51	86.57 ± 4.19	**99.67 ± 0.26**
6	94.08 ± 4.41	95.96 ± 1.91	96.54 ± 1.82	69.27 ± 33.71	72.17 ± 28.85	82.54 ± 5.46	97.49 ± 1.65	**98.13 ± 0.61**
8	98.12 ± 3.03	99.80 ± 0.34	97.64 ± 2.28	98.08 ± 1.84	86.33 ± 9.98	96.38 ± 1.04	99.83 ± 0.35	**100.00 ± 0.00**
10	68.74 ± 1.85	69.43 ± 6.31	67.14 ± 8.06	68.15 ± 15.30	25.90 ± 24.39	68.61 ± 3.72	76.45 ± 4.34	**94.67 ± 1.09**
11	50.07 ± 11.53	62.00 ± 11.48	55.30 ± 5.22	41.86 ± 22.82	37.80 ± 24.74	50.36 ± 6.83	63.15 ± 6.02	**85.62 ± 1.22**
12	67.92 ± 7.26	68.32 ± 6.68	60.38 ± 5.93	48.62 ± 21.57	17.73 ± 20.96	48.20 ± 2.73	76.89 ± 6.46	**97.90 ± 0.42**
13	99.35 ± 1.05	99.84 ± 0.35	99.35 ± 1.05	94.27 ± 5.67	76.22 ± 33.88	99.24 ± 0.73	**99.57 ± 0.86**	99.45 ± 0.28
14	81.04 ± 2.70	93.52 ± 2.29	80.61 ± 2.36	85.29 ± 5.69	88.05 ± 7.80	73.48 ± 5.25	85.22 ± 3.21	**95.94 ± 1.04**
15	85.30 ± 4.72	84.51 ± 14.12	86.56 ± 8.02	66.99 ± 30.71	38.25 ± 20.15	70.98 ± 6.85	92.51 ± 4.87	**99.54 ± 0.37**
16	99.45 ± 0.67	98.77 ± 1.43	96.71 ± 4.63	98.63 ± 1.23	95.07 ± 4.71	99.45 ± 1.10	99.73 ± 0.55	**100.00 ± 0.00**
OA (%)	67.08 ± 2.44	74.88 ± 2.93	70.36 ± 3.98	61.48 ± 9.89	44.46 ± 5.39	60.91 ± 2.44	78.13 ± 1.84	**93.58 ± 0.44**
AA (%)	78.60 ± 1.15	83.19 ± 2.20	80.61 ± 2.37	71.15 ± 8.74	49.97 ± 5.28	70.40 ± 1.55	86.30 ± 0.78	**96.96 ± 0.19**
KA	0.63 ± 0.02	0.72 ± 0.03	0.67 ± 0.04	0.57 ± 0.11	0.38 ± 0.05	0.56 ± 0.03	0.75 ± 0.02	**0.93 ± 0.00**

TABLE VI
CLASSIFICATION RESULTS OF DIFFERENT METHODS ON PU DATASET WITH 20 TRAINING SAMPLES

Class	M3D-DCNN [67]	HybridSN [68]	DFFN [69]	RSSAN [70]	AB-LSTM [71]	SpectralFormer [73]	GAHT [72]	Proposed
1	78.43 ± 18.64	78.84 ± 9.21	79.16 ± 3.33	71.64 ± 8.21	65.37 ± 8.83	60.47 ± 10.05	85.42 ± 6.23	**98.68 ± 0.87**
2	82.56 ± 6.06	81.43 ± 9.31	63.11 ± 5.84	63.53 ± 16.25	39.21 ± 15.41	64.38 ± 10.26	87.99 ± 7.54	**99.61 ± 0.21**
3	88.04 ± 4.37	55.63 ± 9.87	53.36 ± 11.63	58.50 ± 19.99	60.77 ± 16.68	61.12 ± 18.86	91.51 ± 6.25	**97.27 ± 1.49**
4	95.47 ± 4.47	93.40 ± 3.35	87.95 ± 10.90	96.35 ± 2.10	86.80 ± 7.18	89.45 ± 2.03	97.56 ± 1.12	**98.70 ± 0.48**
5	99.83 ± 0.20	99.58 ± 0.47	99.80 ± 0.26	99.50 ± 0.67	99.31 ± 0.29	99.83 ± 0.24	99.98 ± 0.03	**100.00 ± 0.00**
6	73.77 ± 15.38	74.05 ± 13.09	73.17 ± 11.61	79.84 ± 5.67	73.08 ± 10.86	69.41 ± 13.12	94.94 ± 2.16	**98.88 ± 1.24**
7	94.90 ± 2.50	84.52 ± 5.74	75.37 ± 11.69	72.99 ± 26.35	88.43 ± 8.32	75.07 ± 7.45	99.10 ± 0.52	**99.85 ± 0.27**
8	72.09 ± 7.25	73.60 ± 14.09	70.44 ± 14.26	53.10 ± 21.76	66.73 ± 12.13	60.12 ± 14.22	83.37 ± 6.50	**96.13 ± 1.36**
9	99.53 ± 0.46	97.67 ± 1.87	99.16 ± 0.76	98.19 ± 3.36	99.40 ± 0.44	97.37 ± 0.77	99.61 ± 0.20	**100.00 ± 0.00**
OA (%)	82.46 ± 3.67	80.10 ± 5.04	71.02 ± 2.78	70.07 ± 7.99	58.77 ± 6.63	67.78 ± 4.60	89.84 ± 2.82	**98.93 ± 0.20**
AA (%)	87.18 ± 2.27	82.08 ± 3.13	77.95 ± 4.85	77.07 ± 6.00	75.45 ± 1.66	75.25 ± 2.39	93.28 ± 0.69	**98.79 ± 0.21**
KA	0.77 ± 0.04	0.74 ± 0.06	0.64 ± 0.04	0.63 ± 0.09	0.51 ± 0.06	0.60 ± 0.05	0.87 ± 0.03	**0.99 ± 0.00**

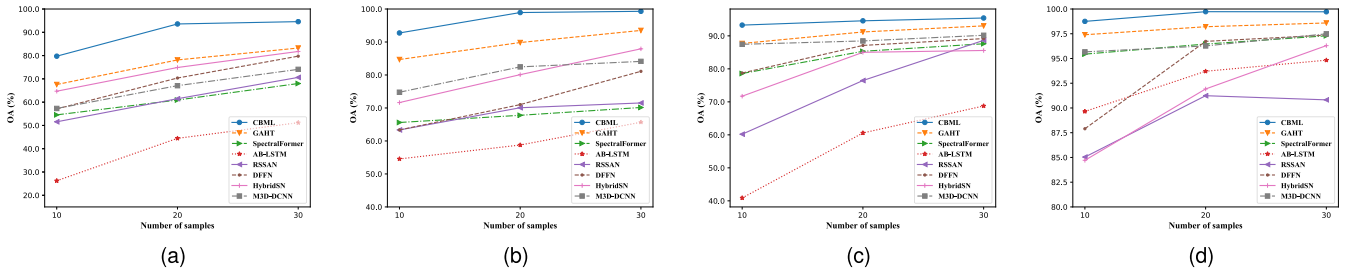


Fig. 6. Classification results with OA of different methods on four benchmark datasets when using 10, 20, and 30 as training samples. (a) IP. (b) PU. (c) SA. (d) PC.

samples, respectively. It can be seen that our proposed method gets excellent results using only 20 training samples for various challenging land cover classes, which demonstrates the effectiveness of our CBML. Figs. 7–10 display the classification maps of IP, PU, SA, and PC datasets with 30 training samples, respectively. From the figures, we can find that the samples in the boundary of two land cover categories can be accurately identified compared to other methods, owing to the ability of our CBML to pull the samples to their corresponding feature centers.

E. Comparison With Well-Known Metric Loss Functions

In order to demonstrate the performance of CBML, we compare CBML with three well-known metric loss functions, including contrastive loss, triplet loss, and multisimilarity loss. To show the superiority of metric learning, we also compare metric losses with the cross-entropy loss; 10 and 20 cubes with the spatial size of 7×7 are randomly selected from four datasets as training data. In these experiments, cross mini-batch sampling strategy is not used.

Fig. 11 shows OA, AA, and KA for the five loss functions on IP, PU, SA, and PC datasets when using ten samples as training data. Tables IX–XII show the classification results of all the loss functions on four benchmark datasets when using 20 training samples. From the results, we can easily find that deep metric loss functions, except contrastive loss, have significantly improved the three classification indicators of OA, AA, and KA compared with the traditional cross-entropy

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:37:13 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
CLASSIFICATION RESULTS OF DIFFERENT METHODS ON SA DATASET WITH 20 TRAINING SAMPLES

Class	M3D-DCNN [67]	HybridSN [68]	DFFN [69]	RSSAN [70]	AB-LSTM [71]	SpectralFormer [73]	GAHT [72]	Proposed
1	98.22 ± 1.39	95.69 ± 11.87	98.32 ± 0.90	99.87 ± 0.13	97.22 ± 2.86	95.78 ± 0.85	97.30 ± 4.04	**99.98 ± 0.03**
2	94.39 ± 9.91	88.00 ± 25.51	98.83 ± 0.30	71.34 ± 38.96	94.64 ± 3.89	90.80 ± 8.48	99.84 ± 0.23	**100.00 ± 0.00**
3	90.96 ± 6.09	90.78 ± 5.04	92.90 ± 4.65	85.40 ± 21.10	47.46 ± 33.03	90.22 ± 5.20	99.31 ± 0.79	**99.97 ± 0.05**
4	99.02 ± 0.70	99.75 ± 0.50	97.50 ± 1.44	97.41 ± 2.85	97.02 ± 2.09	96.10 ± 1.88	**99.53 ± 0.42**	99.48 ± 0.22
5	95.64 ± 4.19	98.78 ± 2.11	91.65 ± 5.64	95.52 ± 2.31	59.19 ± 41.37	80.59 ± 5.54	96.13 ± 1.90	**99.94 ± 0.05**
6	99.68 ± 0.27	99.78 ± 0.38	99.09 ± 0.71	98.65 ± 1.24	72.87 ± 37.89	99.24 ± 0.46	99.67 ± 0.44	**100.00 ± 0.00**
7	99.47 ± 0.39	91.43 ± 8.21	99.33 ± 0.76	94.86 ± 5.58	98.08 ± 1.81	97.39 ± 1.54	99.90 ± 0.08	**100.00 ± 0.00**
8	75.50 ± 8.63	59.62 ± 8.17	73.18 ± 10.04	40.00 ± 30.64	35.61 ± 19.54	70.28 ± 13.84	71.97 ± 10.04	**81.83 ± 2.98**
9	97.22 ± 2.43	99.83 ± 0.18	93.15 ± 4.15	98.02 ± 1.80	51.44 ± 42.03	92.67 ± 2.09	99.34 ± 0.57	**99.90 ± 0.22**
10	87.97 ± 4.00	88.54 ± 23.44	79.85 ± 5.42	66.08 ± 24.54	17.27 ± 21.84	76.54 ± 10.03	93.03 ± 1.54	**99.08 ± 0.20**
11	94.47 ± 4.81	97.24 ± 1.21	94.20 ± 5.47	93.23 ± 5.48	25.95 ± 21.25	90.27 ± 2.90	99.03 ± 0.72	**99.90 ± 0.10**
12	98.97 ± 1.36	91.01 ± 20.53	99.14 ± 0.93	89.90 ± 14.89	55.29 ± 45.59	98.42 ± 0.76	99.99 ± 0.02	**100.00 ± 0.00**
13	99.04 ± 0.98	92.96 ± 14.61	98.77 ± 1.10	62.32 ± 44.44	78.17 ± 28.31	98.08 ± 1.91	99.96 ± 0.05	**100.00 ± 0.00**
14	95.85 ± 2.78	89.34 ± 23.10	97.81 ± 1.42	75.54 ± 37.86	64.36 ± 33.03	96.32 ± 3.21	98.93 ± 0.70	**98.88 ± 0.38**
15	71.56 ± 11.91	77.27 ± 21.86	72.30 ± 9.77	74.82 ± 16.04	75.84 ± 11.84	76.34 ± 9.63	85.36 ± 7.39	**89.13 ± 2.76**
16	93.34 ± 2.51	96.81 ± 2.41	89.33 ± 6.96	85.19 ± 11.81	56.31 ± 45.54	92.78 ± 2.61	97.62 ± 1.55	**99.16 ± 0.37**
OA (%)	88.46 ± 1.50	85.01 ± 7.67	87.10 ± 2.02	76.46 ± 9.38	60.57 ± 9.67	85.32 ± 2.28	91.18 ± 1.33	**94.56 ± 0.52**
AA (%)	93.21 ± 1.18	91.05 ± 9.27	92.21 ± 1.03	83.01 ± 11.03	64.17 ± 10.99	90.11 ± 1.08	96.06 ± 0.33	**97.95 ± 0.17**
KA	0.87 ± 0.02	0.83 ± 0.08	0.86 ± 0.02	0.74 ± 0.10	0.57 ± 0.10	0.84 ± 0.02	0.90 ± 0.01	**0.94 ± 0.01**

TABLE VIII
CLASSIFICATION RESULTS OF DIFFERENT METHODS ON PC DATASET WITH 20 TRAINING SAMPLES

Class	M3D-DCNN [67]	HybridSN [68]	DFFN [69]	RSSAN [70]	AB-LSTM [71]	SpectralFormer [73]	GAHT [72]	Proposed
1	99.15 ± 0.41	99.42 ± 0.25	99.33 ± 0.40	99.55 ± 0.46	99.70 ± 0.43	99.44 ± 0.31	99.84 ± 0.21	**100.00 ± 0.00**
2	93.60 ± 3.38	85.84 ± 4.68	89.82 ± 4.20	90.95 ± 6.08	92.19 ± 3.49	91.70 ± 2.86	94.15 ± 4.42	**98.56 ± 0.47**
3	88.02 ± 10.41	77.07 ± 24.33	88.96 ± 5.78	83.96 ± 9.89	86.21 ± 4.68	85.52 ± 12.78	89.25 ± 9.33	**98.92 ± 0.91**
4	89.73 ± 13.11	79.34 ± 27.06	95.96 ± 3.53	84.03 ± 13.45	87.58 ± 4.86	95.87 ± 3.64	96.10 ± 3.47	**99.86 ± 0.07**
5	88.69 ± 4.74	76.06 ± 25.02	84.91 ± 3.89	91.32 ± 5.40	69.88 ± 17.53	84.58 ± 5.28	90.02 ± 7.16	**99.49 ± 0.72**
6	93.98 ± 3.92	90.19 ± 5.79	95.93 ± 4.49	97.54 ± 1.25	87.70 ± 2.31	90.45 ± 2.49	97.84 ± 1.35	**99.92 ± 0.04**
7	86.28 ± 2.57	78.37 ± 25.92	85.50 ± 1.97	87.00 ± 3.52	79.35 ± 3.43	86.85 ± 1.53	91.63 ± 2.99	**97.93 ± 0.19**
8	96.41 ± 1.75	87.97 ± 14.94	98.40 ± 0.61	78.27 ± 37.22	93.96 ± 3.63	98.36 ± 0.98	99.53 ± 0.41	**99.85 ± 0.15**
9	100.00 ± 0.00	98.36 ± 0.55	97.37 ± 2.45	99.26 ± 0.67	80.04 ± 39.53	95.48 ± 4.74	99.89 ± 0.09	**100.00 ± 0.00**
OA (%)	96.27 ± 0.79	91.92 ± 7.60	96.73 ± 0.29	91.24 ± 10.94	93.71 ± 1.68	96.46 ± 0.42	98.21 ± 0.42	**99.73 ± 0.09**
AA (%)	92.87 ± 0.78	85.85 ± 12.75	92.91 ± 0.71	90.21 ± 5.27	86.29 ± 6.61	92.03 ± 1.36	95.36 ± 1.03	**99.39 ± 0.19**
KA	0.95 ± 0.01	0.89 ± 0.10	0.95 ± 0.00	88.37 ± 0.13	0.91 ± 0.02	0.95 ± 0.00	0.97 ± 0.00	**1.00 ± 0.00**

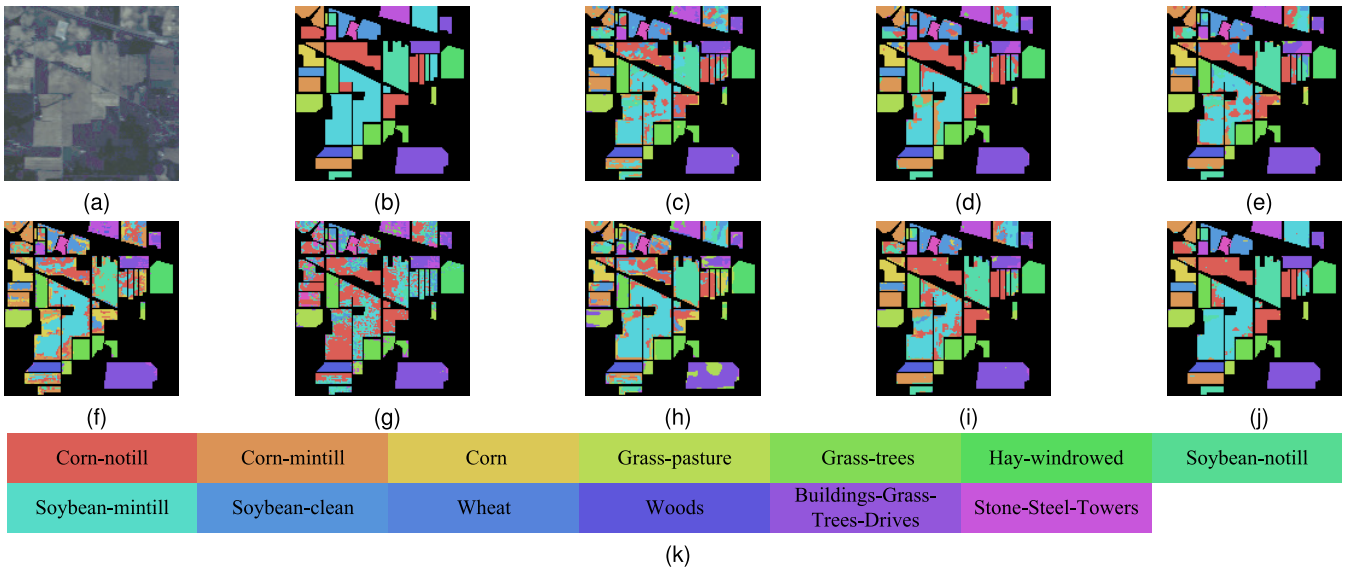


Fig. 7. Classification maps of IP dataset with 30 training samples. (a) False-color image. (b) Ground truth. (c) HybridSN. (d) M3D-DCNN. (e) DFFN. (f) RSSAN. (g) AB-LSTM. (h) SpectralFormer. (i) GAHT. (j) Proposed method. (k) Legend of different classes.

loss, which proves the effectiveness of the metric learning in HSI classification. In particular, the proposed CBML shows significant superiority over other metric losses. For example, our CBML achieves the best OA, AA, and KA on SA and IP

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:37:13 UTC from IEEE Xplore. Restrictions apply.

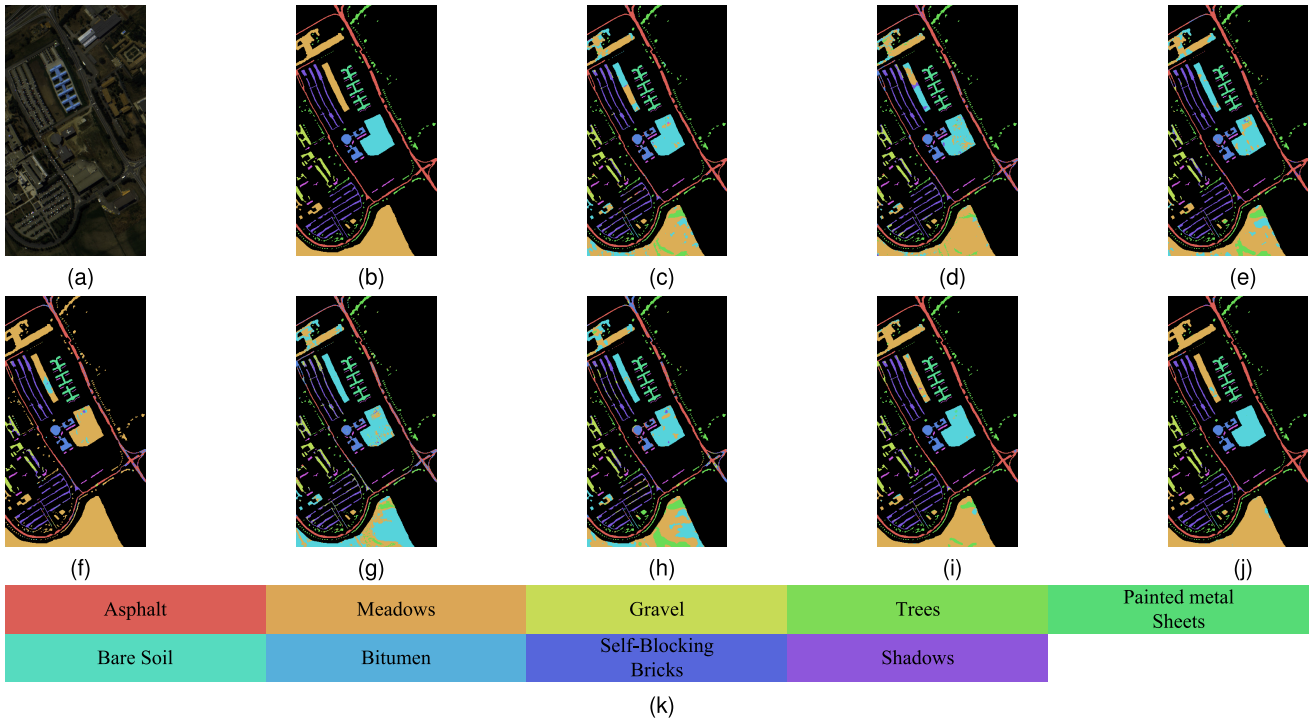


Fig. 8. Classification maps of PU dataset with 30 training samples. (a) False-color image. (b) Ground truth. (c) HybridSN. (d) M3D-DCNN. (e) DFFN. (f) RSSAN. (g) AB-LSTM. (h) SpectralFormer. (i) GAHT. (j) Proposed method. (k) Legend of different classes.

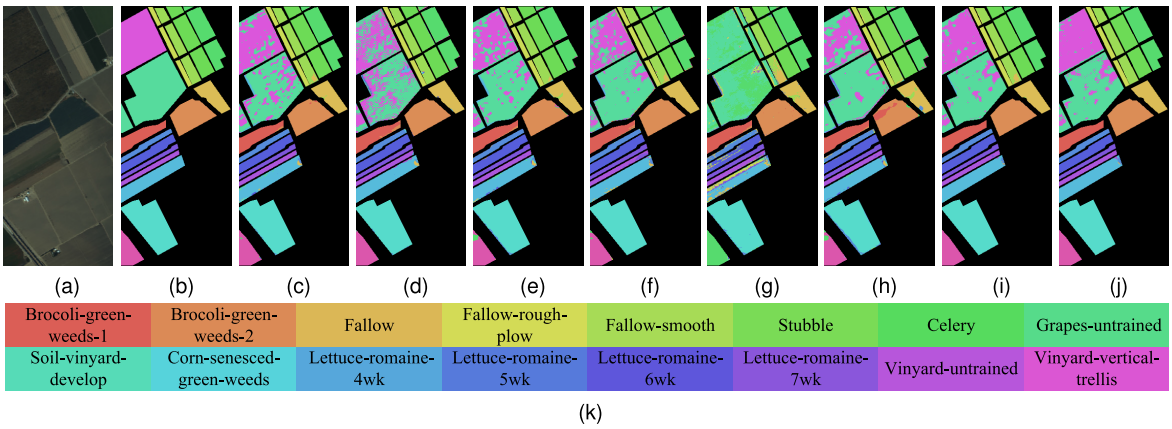


Fig. 9. Classification maps of SA dataset with 30 training samples. (a) False-color image. (b) Ground truth. (c) HybridSN. (d) M3D-DCNN. (e) DFFN. (f) RSSAN. (g) AB-LSTM. (h) SpectralFormer. (i) GAHT. (j) Proposed method. (k) Legend of different classes.

TABLE IX

CLASSIFICATION RESULTS OF DIFFERENT LOSS FUNCTIONS WHEN USING 20 SAMPLES AS TRAINING DATA ON IP DATASET

	CrossEntropy	Contrastive [44]	Triplet [45]	Multi-similarity [46]	CBML
OA (%)	77.66 ± 1.31	77.47 ± 1.38	88.73 ± 1.63	**90.07 ± 1.48**	89.92 ± 1.51
AA (%)	85.58 ± 0.99	86.82 ± 0.78	93.72 ± 0.73	94.87 ± 0.70	**94.9 ± 0.77**
KA	0.75± 0.01	0.75± 0.02	0.87± 0.02	0.89± 0.02	**0.89± 0.02**

TABLE X

CLASSIFICATION RESULTS OF DIFFERENT LOSS FUNCTIONS WHEN USING 20 SAMPLES AS TRAINING DATA ON PU DATASET

	CrossEntropy	Contrastive [44]	Triplet [45]	Multi-similarity [46]	CBML
OA (%)	91.46± 1.24	92.86± 1.30	97.40± 0.43	97.93± 0.24	**98.64± 0.24**
AA (%)	94.71± 0.62	94.28± 0.48	97.75± 0.35	98.04± 0.15	**98.99± 0.15**
KA	0.89± 0.02	0.91± 0.02	0.97± 0.01	0.97± 0.00	**0.98± 0.00**

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:37:13 UTC from IEEE Xplore. Restrictions apply.

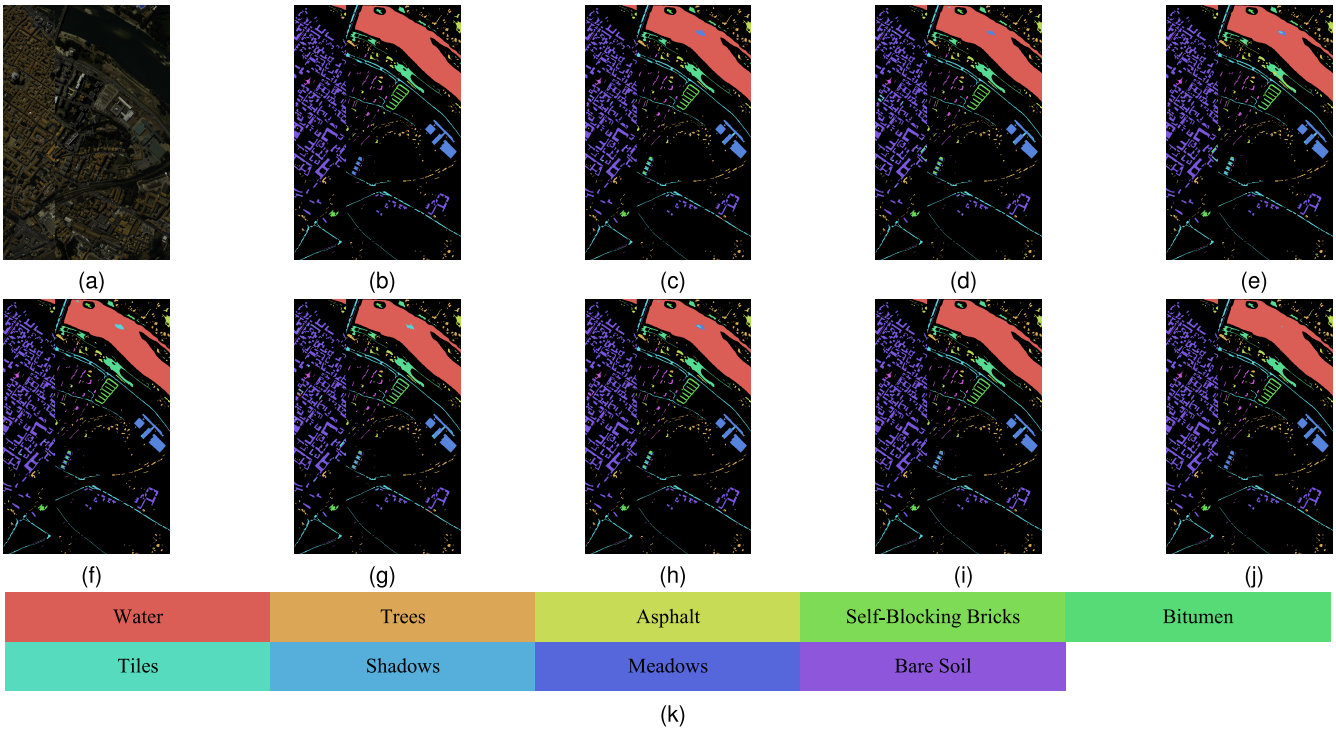


Fig. 10. Classification maps of PC dataset with 30 training samples. (a) False-color image. (b) Ground truth. (c) HybridSN. (d) M3D-DCNN. (e) DFFN. (f) RSSAN. (g) AB-LSTM. (h) SpectralFormer. (i) GAHT. (j) Proposed method. (k) Legend of different classes.

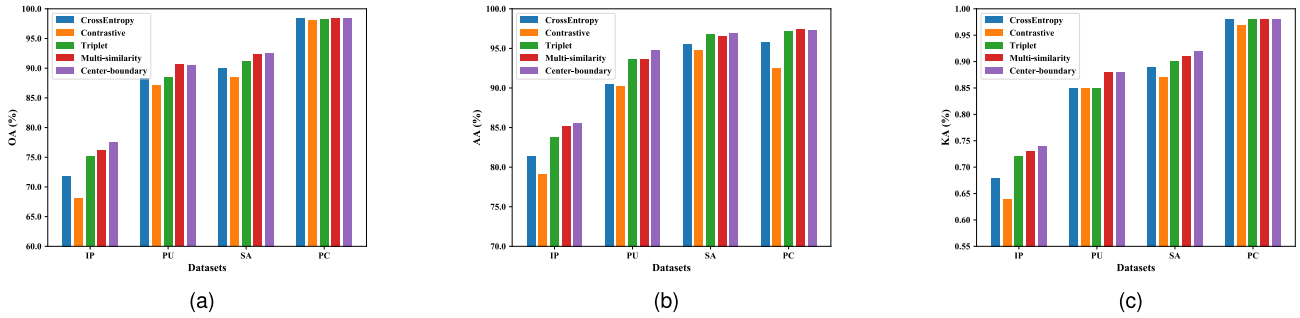


Fig. 11. Classification results of different loss functions on four benchmark datasets when using ten samples as training data. (a) OA. (b) AA. (c) KA.

TABLE XI
CLASSIFICATION RESULTS OF DIFFERENT LOSS FUNCTIONS WHEN USING 20 SAMPLES AS TRAINING DATA ON SA DATASET

	CrossEntropy	Contrastive [44]	Triplet [45]	Multi-similarity [46]	CBML
OA (%)	92.31± 0.61	87.14± 0.77	93.12± 0.45	**94.20± 1.27**	93.98± 0.45
AA (%)	96.80± 0.25	94.89± 0.27	97.30± 0.32	97.81± 0.38	**97.82± 0.12**
KA	0.91± 0.01	0.86± 0.01	0.92± 0.00	**0.94± 0.01**	0.93± 0.00

TABLE XII
CLASSIFICATION RESULTS OF DIFFERENT LOSS FUNCTIONS WHEN USING 20 SAMPLES AS TRAINING DATA ON PC DATASET

	CrossEntropy	Contrastive [44]	Triplet [45]	Multi-similarity [46]	CBML
OA (%)	98.49± 0.20	99.13± 0.20	98.88± 0.11	99.36± 0.18	**99.67± 0.05**
AA (%)	97.41± 0.38	98.37± 0.46	97.24± 0.30	99.00± 0.12	**99.19± 0.17**
KA	0.98± 0.00	0.99± 0.00	0.98± 0.00	0.99± 0.00	**1.00± 0.00**

datasets, and yields similar best results on PC and PU datasets, when using ten training samples. With respect to 20 samples as training data, CBML also shows its effectiveness, including best results on PC and PU datasets, and comparable well performance with multisimilarity loss on IP and PC datasets.

Contrastive loss only focuses the distance within the input positive and negative sample pairs, but ignores the relationship

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:37:13 UTC from IEEE Xplore. Restrictions apply.

TABLE XIII
PARAMETERS AND TIME CONSUMPTION OF DIFFERENT LOSS FUNCTIONS TO TRAIN SSRN WITH TEN TRAINING SAMPLES ON IP DATASET

	CrossEntropy	Contrastive [44]	Triplet [45]	Multi-similarity [46]	CBML
Parameters (k)	364.093	363.768	363.768	363.768	364.080
Time (s)	1.563± 0.015	1.567± 0.005	1.553± 0.004	1.569 ± 0.010	1.581± 0.015

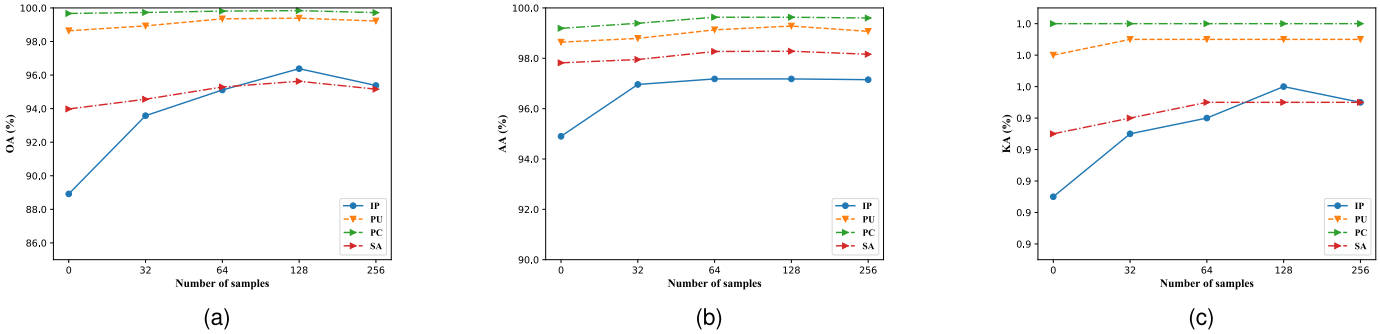


Fig. 12. Classification results of different queue lengths on four benchmark datasets when using 20 samples as training data. (a) OA. (b) AA. (c) KA.

between samples, resulting in slow model convergence. Triplet loss proposes a triplet to further extract the information of sample pairs and employs the hard triplet mining strategy to facilitate model convergence. However, only the hardest sample exemplars are selected, resulting in training data not being fully exploited. Multisimilarity loss not only calculates the distance of the hardest sample pairs but also considers the relationship between the hardest sample pairs and their surrounding sample pairs to fully exploit the informative information in the training data. Nonetheless, it ignores the relationship between samples and their feature centers. On the contrary, CBML is capable of pulling the distance of the sample to its feature center closer, effectively enhancing the discrimination of features extracted by the model, and thus improving the classification results.

Table XIII displays a comparison of parameters and time consumption of different loss functions to train a model for one epoch with ten samples as training data on the IP dataset. It can be seen that cross-entropy loss has the most model parameters because of an additional classification network. Due to the use of a linear neural network without bias to compute the distance of samples to their corresponding feature centers, our proposed CBML has more model parameters compared to other metric loss functions. Furthermore, CBML requires more computational resources due to its complex distance calculations between samples and feature centers. Nevertheless, employing CBML to train the model is an end-to-end process without the requirement to train an additional classifier like other metric losses, which significantly reduces training time. Moreover, the class of a sample can be directly determined by its distance to class feature centers in the testing process. The additional time consumption is insignificant compared with cross-entropy loss. Generally, our approach makes a tradeoff between performance and computational resources.

F. Contribution of Cross-Mini-Batch Sampling Strategy

To prove the effectiveness of our proposed cross-mini-batch sampling strategy, we also do adequate ablation experiments in HSI classification. In our experiments, we take different lengths of the queue to show their effect on the classification results. We randomly select 20 cubes with the spatial size of 7×7 from four benchmark datasets as training samples. In the training time, we take queues with five different lengths to store feature vectors of samples from several cross mini-batches, and the size of queues is set as 0, 32, 64, 128, and 256, respectively, where 0 means cross mini-batch sampling strategy is not used.

Fig. 12 displays the classification results with OA, AA, and KA on four HSI datasets. As shown in the figure, it can be seen that when the length of the queue is 0, the performance of our CBML is not optimal on all datasets, which reveals that our proposed sampling strategy can effectively increase the sampling range and make full use of the training data, so as to improve the classification results. In addition, all the metric values show a trend of first increasing and then decreasing as the size of the queue length expands on four datasets, which indicates that the length of the queue cannot be set too large. The possible reason for this phenomenon is that when the queue length is too large, many sample pairs that do not contribute or contribute little to the loss functions are sampled, which would affect the convergence of the model, thereby reducing the performance. In general, our proposed cross mini-batch sampling strategy breaks through the limitation of sampling range within a mall mini-batch, takes full advantage of informative information in training data, and further improves the performance of CBML.

G. Extensibility to Other Backbone Network

In order to demonstrate the extensibility of our proposed CBML to other backbone networks, another model, GAHT [72], is also selected as the base network and a series of experiments are conducted on four datasets. 10, 20, and 30 cubes with a spatial size of 7×7 as training data, and the length of the queue is set to 32.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:37:13 UTC from IEEE Xplore. Restrictions apply.

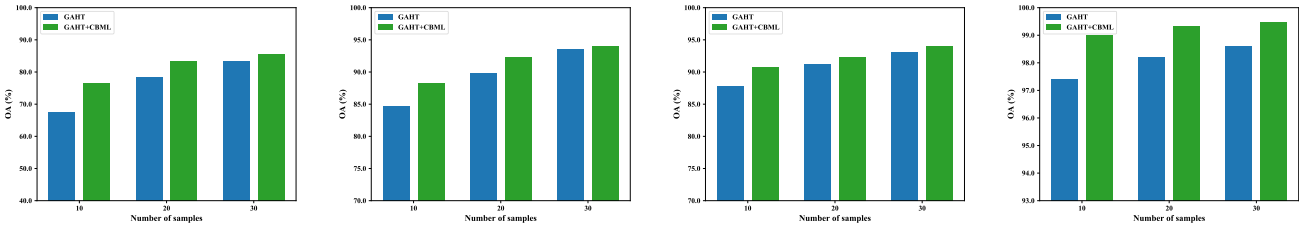


Fig. 13. Comparison results with OA of GAHT and GAHT + CBML on four benchmark datasets. (a) IP. (b) PU. (c) SA. (d) PC.

As can be seen in Fig. 13, after applying our proposed method on GAHT, the classification results have been improved on each dataset with three different numbers of training samples, which fully proves the effectiveness of our CBML. Notably, GAHT is a transformer-based network, and the above results provide strong evidence for the extensibility of our proposed CBML.

V. Conclusion

In this article, we have proposed a new metric loss function named CBML to enhance the discriminative features of samples in the embedding feature space. CBML attempts to force the distance of a sample to its counterpart feature center to be definitely smaller than to other samples from different classes by a predefined threshold. Then, the boundaries of different classes will explicitly interval a certain distance. We also present a new approach that adopts a linear neural network to learn the distance of a sample to its corresponding feature center instead of using parameter centers. In this way, it can effectively alleviate computational consumption and instability of the training process. To further improve training efficiency, we introduce the pair mining and weighting strategy to CBML so as to make full use of information in all sample pairs. Moreover, the class of samples is capable of being determined by the distance to class feature centers due to the feature centers have been learned during the training process.

In order to expand the range of sample pairs as well as fully explore informative information in training data, we design a cross mini-batch sampling strategy to improve the memory bank-based method based on "feature similarity" phenomenon. The cross mini-batch sampling strategy constructs a queue to save the feature vectors of samples from several adjacent mini-batches and combine them with the current mini-batch to sample pairs. Therefore, the range of sample pairs has obtained a great improvement so as to break the limit of sample pairs within a small mini-batch. The cross-batch sampling strategy suggested in this article operates effectively in the presence of "progressive feature similarity." Nevertheless, it is empirical to determine how to assess the appearance of this similarity. In the future, a quantitative index for measuring the phenomenon of "progressive feature similarity" deserves to be further researched.

References

[1] B. Chen, B. Huang, and B. Xu, "Multi-source remotely sensed data fusion for improving land cover classification," *ISPRS J. Photogramm. Remote Sens.*, vol. 124, pp. 27–39, Feb. 2017.

[2] J. M. Bioucas-Dias, A. Plaza, G. Camps-Valls, P. Scheunders, N. Nasrabadi, and J. Chanussot, "Hyperspectral remote sensing data analysis and future challenges," *IEEE Geosci. Remote Sens. Mag.*, vol. 1, no. 2, pp. 6–36, Jun. 2013.

[3] X. Zhang, G. Wen, and W. Dai, "A tensor decomposition-based anomaly detection algorithm for hyperspectral image," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 10, pp. 5801–5820, Oct. 2016.

[4] R. Zhao, B. Du, and L. Zhang, "Hyperspectral anomaly detection via a sparsity score estimation framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 6, pp. 3208–3222, Jun. 2017.

[5] Y. Li, H. Zhang, and Q. Shen, "Spectral–spatial classification of hyperspectral imagery with 3D convolutional neural network," *Remote Sens.*, vol. 9, no. 1, p. 67, Jan. 2017.

[6] X. Ma, H. Wang, and J. Geng, "Spectral–spatial classification of hyperspectral image based on deep auto-encoder," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 9, no. 9, pp. 4073–4085, Sep. 2016.

[7] S. Mei, J. Ji, J. Hou, X. Li, and Q. Du, "Learning sensor-specific spatial–spectral features of hyperspectral images via convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 8, pp. 4520–4533, Aug. 2017.

[8] S. Mei, J. Ji, Y. Geng, Z. Zhang, X. Li, and Q. Du, "Unsupervised spatial–spectral feature learning by 3D convolutional autoencoder for hyperspectral classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 9, pp. 6808–6820, Sep. 2019.

[9] Y. Zhang, W. Li, M. Zhang, S. Wang, R. Tao, and Q. Du, "Graph information aggregation cross-domain few-shot learning for hyperspectral image classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 2, pp. 1912–1925, Feb. 2024.

[10] K. Makantasis, K. Karantzalos, A. Doulamis, and N. Doulamis, "Deep supervised learning for hyperspectral data classification through convolutional neural networks," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Jul. 2015, pp. 4959–4962.

[11] X. He, Y. Chen, and Z. Lin, "Spatial–spectral transformer for hyperspectral image classification," *Remote Sens.*, vol. 13, no. 3, p. 498, Jan. 2021.

[12] X. Tan, K. Gao, B. Liu, Y. Fu, and L. Kang, "Deep global-local transformer network combined with extended morphological profiles for hyperspectral image classification," *J. Appl. Remote Sens.*, vol. 15, no. 3, Sep. 2021, Art. no. 038509.

[13] J. He, L. Zhao, H. Yang, M. Zhang, and W. Li, "HSI-BERT: Hyperspectral image classification using the bidirectional encoder representation from transformers," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 1, pp. 165–178, Jan. 2020.

[14] X. Yang, W. Cao, Y. Lu, and Y. Zhou, "Hyperspectral image transformer classification networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5528715.

[15] F. Xu, G. Zhang, C. Song, H. Wang, and S. Mei, "Multiscale and cross-level attention learning for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5501615.

[16] G. Chechik, U. Shalit, V. Sharma, and S. Bengio, "An online algorithm for large scale image similarity learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 22, 2009, pp. 306–314.

[17] B. Kulis, M. A. Sustik, and I. S. Dhillon, "Low-rank kernel learning with Bregman matrix divergences," *J. Mach. Learn. Res.*, vol. 10, no. 2, pp. 1–36, 2009.

[18] B. Kulis, "Metric learning: A survey," *Found. Trends Mach. Learn.*, vol. 5, no. 4, pp. 287–364, 2013.

[19] E. Xing, M. Jordan, S. J. Russell, and A. Ng, "Distance metric learning with application to clustering with side-information," *Adv. Neural Inf. Process. Syst.*, vol. 15, 2002, pp. 521–528.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:37:13 UTC from IEEE Xplore. Restrictions apply.

[20] K. Q. Weinberger and L. K. Saul, "Distance metric learning for large margin nearest neighbor classification," *J. Mach. Learn. Res.*, vol. 10, no. 2, pp. 1–38, 2009.

[21] B. Geng, D. Tao, and C. Xu, "DAML: Domain adaptation metric learning," *IEEE Trans. Image Process.*, vol. 20, no. 10, pp. 2980–2989, Oct. 2011.

[22] S. C. H. Hoi, W. Liu, and S.-F. Chang, "Semi-supervised distance metric learning for collaborative image retrieval and clustering," *ACM Trans. Multimedia Comput., Commun., Appl.*, vol. 6, no. 3, pp. 1–26, Aug. 2010.

[23] E. Ustinova and V. Lempitsky, "Learning deep embeddings with histogram loss," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 29, 2016, pp. 4170–4178.

[24] G. Dai, J. Xie, and Y. Fang, "Deep correlated holistic metric learning for sketch-based 3D shape retrieval," *IEEE Trans. Image Process.*, vol. 27, no. 7, pp. 3374–3386, Jul. 2018.

[25] R. Cao et al., "Enhancing remote sensing image retrieval using a triplet deep metric learning network," *Int. J. Remote Sens.*, vol. 41, no. 2, pp. 740–751, Jan. 2020.

[26] Z. Ding and Y. Fu, "Robust transfer metric learning for image classification," *IEEE Trans. Image Process.*, vol. 26, no. 2, pp. 660–670, Feb. 2017.

[27] Y. Luo, T. Liu, D. Tao, and C. Xu, "Decomposition-based transfer distance metric learning for image classification," *IEEE Trans. Image Process.*, vol. 23, no. 9, pp. 3789–3801, Sep. 2014.

[28] G. Cheng, P. Zhou, and J. Han, "Duplex metric learning for image set classification," *IEEE Trans. Image Process.*, vol. 27, no. 1, pp. 281–292, Oct. 2017.

[29] Y. Zheng, J. Fan, J. Zhang, and X. Gao, "Exploiting related and unrelated tasks for hierarchical metric learning and image classification," *IEEE Trans. Image Process.*, vol. 29, pp. 883–896, 2020.

[30] G. Cheng, C. Yang, X. Yao, L. Guo, and J. Han, "When deep learning meets metric learning: Remote sensing image scene classification via learning discriminative CNNs," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 5, pp. 2811–2821, May 2018.

[31] Y. Wen, K. Zhang, Z. Li, and Y. Qiao, "A discriminative feature learning approach for deep face recognition," in *Proc. Eur. Conf. Comput. Vision. (ECCV)*. Cham, Switzerland: Springer, 2016, pp. 499–515.

[32] J. Wang, F. Zhou, S. Wen, X. Liu, and Y. Lin, "Deep metric learning with angular loss," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2593–2601.

[33] J. Lu, J. Hu, and Y.-P. Tan, "Discriminative deep metric learning for face and kinship verification," *IEEE Trans. Image Process.*, vol. 26, no. 9, pp. 4269–4282, Sep. 2017.

[34] J. Snell, K. Swersky, and R. Zemel, "Prototypical networks for few-shot learning," in *Proc. Adv. neural Inf. Process. Syst.*, vol. 30, 2017, pp. 4077–4087.

[35] F. Sung, Y. Yang, L. Zhang, T. Xiang, P. H. S. Torr, and T. M. Hospedales, "Learning to compare: Relation network for few-shot learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Salt Lake City, UT, USA, Jun. 2018, pp. 1199–1208.

[36] W. Jiang, K. Huang, J. Geng, and X. Deng, "Multi-scale metric learning for few-shot learning," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 3, pp. 1091–1102, Mar. 2021.

[37] R. Tao, E. Gavves, and A. W. M. Smeulders, "Siamese instance search for tracking," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 1420–1429.

[38] Q. Guo, W. Feng, C. Zhou, R. Huang, L. Wan, and S. Wang, "Learning dynamic Siamese network for visual object tracking," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 1763–1771.

[39] Z. Tu, C. Lin, W. Zhao, C. Li, and J. Tang, "M5L: Multi-modal multi-margin metric learning for RGBT tracking," *IEEE Trans. Image Process.*, vol. 31, pp. 85–98, 2022.

[40] D. Yi, Z. Lei, S. Liao, and S. Z. Li, "Deep metric learning for person re-identification," in *Proc. 22nd Int. Conf. Pattern Recognit.*, Aug. 2014, pp. 34–39.

[41] H. Shi et al., "Embedding deep metric for person re-identification: A study against large variations," in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, 2016, pp. 732–748.

[42] X. Yang, M. Wang, and D. Tao, "Person re-identification with metric learning using privileged information," *IEEE Trans. Image Process.*, vol. 27, no. 2, pp. 791–805, Feb. 2018.

[43] D. Tao, Y. Guo, M. Song, Y. Li, Z. Yu, and Y. Y. Tang, "Person re-identification by dual-regularized KISS metric learning," *IEEE Trans. Image Process.*, vol. 25, no. 6, pp. 2726–2738, Jun. 2016.

[44] R. Hadsell, S. Chopra, and Y. LeCun, "Dimensionality reduction by learning an invariant mapping," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2006, pp. 1735–1742.

[45] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 815–823.

[46] X. Wang, X. Han, W. Huang, D. Dong, and M. R. Scott, "Multi-similarity loss with general pair weighting for deep metric learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2019, pp. 5022–5030.

[47] H. O. Song, Y. Xiang, S. Jegelka, and S. Savarese, "Deep metric learning via lifted structured feature embedding," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 4004–4012.

[48] K. Sohn, "Improved deep metric learning with multi-class n-Pair loss objective," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 29, 2016, pp. 1857–1865.

[49] X. Wang, Y. Hua, E. Kodirov, G. Hu, R. Garnier, and N. M. Robertson, "Ranked list loss for deep metric learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 5202–5211.

[50] W. Zuo et al., "Distance metric learning via iterated support vector machines," *IEEE Trans. Image Process.*, vol. 26, no. 10, pp. 4937–4950, Oct. 2017.

[51] J. Bromley, I. Guyon, Y. LeCun, E. Säckinger, and R. Shah, "Signature verification using a 'Siamese,' time delay neural network," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 6, 1993, pp. 737–744.

[52] Z. Liang and J. Shen, "Local semantic Siamese networks for fast tracking," *IEEE Trans. Image Process.*, vol. 29, pp. 3351–3364, 2019.

[53] E. Hoffer and N. Ailon, "Deep metric learning using triplet network,," in *Proc. Int. Workshop Similarity-Based Pattern Recognit. (SIMBAD)*, Copenhagen, Denmark, 2015, pp. 84–92.

[54] G. He, F. Li, Q. Wang, Z. Bai, and Y. Xu, "A hierarchical sampling based triplet network for fine-grained image classification," *Pattern Recognit.*, vol. 115, Jul. 2021, Art. no. 107889.

[55] M. T. Law, N. Thome, and M. Cord, "Quadruplet-wise image similarity learning," in *Proc. IEEE Int. Conf. Comput. Vis.*, Dec. 2013, pp. 249–256.

[56] W. Chen, X. Chen, J. Zhang, and K. Huang, "Beyond triplet loss: A deep quadruplet network for person re-identification," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 403–412.

[57] X. Dong, J. Shen, D. Wu, K. Guo, X. Jin, and F. Porikli, "Quadruplet network with one-shot learning for fast visual object tracking," *IEEE Trans. Image Process.*, vol. 28, no. 7, pp. 3516–3527, Jul. 2019.

[58] B. Nguyen and B. De Baets, "Kernel distance metric learning using pairwise constraints for person re-identification," *IEEE Trans. Image Process.*, vol. 28, no. 2, pp. 589–600, Feb. 2019.

[59] J. Yu, X. Yang, F. Gao, and D. Tao, "Deep multimodal distance metric learning using click constraints for image ranking," *IEEE Trans. Cybern.*, vol. 47, no. 12, pp. 4014–4024, Dec. 2016.

[60] K. H. Jin, M. T. McCann, E. Froustey, and M. Unser, "Deep convolutional neural network for inverse problems in imaging," *IEEE Trans. Image Process.*, vol. 26, no. 9, pp. 4509–4522, Sep. 2017.

[61] S. Bosse, D. Maniry, K.-R. Müller, T. Wiegand, and W. Samek, "Deep neural networks for no-reference and full-reference image quality assessment," *IEEE Trans. Image Process.*, vol. 27, pp. 206–219, 2017.

[62] L. Ma, X. Yang, and D. Tao, "Person re-identification over camera networks using multi-task distance metric learning," *IEEE Trans. Image Process.*, vol. 23, no. 8, pp. 3656–3670, Aug. 2014.

[63] D. Wang and X. Tan, "Robust distance metric learning via Bayesian inference," *IEEE Trans. Image Process.*, vol. 27, no. 3, pp. 1542–1553, Mar. 2018.

[64] Z. Wu, Y. Xiong, S. X. Yu, and D. Lin, "Unsupervised feature learning via non-parametric instance discrimination," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 3733–3742.

[65] X. He, Y. Zhou, Z. Zhou, S. Bai, and X. Bai, "Triplet-center loss for multi-view 3D object retrieval," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 1945–1954.

[66] Z. Zhong, J. Li, Z. Luo, and M. Chapman, "Spectral–spatial residual network for hyperspectral image classification: A 3-D deep learning framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 847–858, Feb. 2017.

[67] M. He, B. Li, and H. Chen, "Multi-scale 3D deep convolutional neural network for hyperspectral image classification," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Sep. 2017, pp. 3904–3908.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:37:13 UTC from IEEE Xplore. Restrictions apply.

[68] S. K. Roy, G. Krishna, S. R. Dubey, and B. B. Chaudhuri, "HybridSN: Exploring 3D-2D CNN feature hierarchy for hyperspectral image classification," *IEEE Geosci. Remote Sens.*, vol. 17, no. 2, pp. 277–281, Feb. 2020.
[69] W. Song, S. Li, L. Fang, and T. Lu, "Hyperspectral image classification with deep feature fusion network," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 6, pp. 3173–3184, Jun. 2018.
[70] M. Zhu, L. Jiao, F. Liu, S. Yang, and J. Wang, "Residual spectral–spatial attention network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 1, pp. 449–462, Jan. 2021.
[71] S. Mei, X. Li, X. Liu, H. Cai, and Q. Du, "Hyperspectral image classification using attention-based bidirectional long short-term memory network," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5509612.
[72] S. Mei, C. Song, M. Ma, and F. Xu, "Hyperspectral image classification using group-aware hierarchical transformer," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5539014.
[73] D. Hong et al., "SpectralFormer: Rethinking hyperspectral image classification with transformers," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5518615.



Shaohui Mei (Senior Member, IEEE) received the B.S. degree in electronics and information engineering and the Ph.D. degree in signal and information processing from Northwestern Polytechnical University, Xi'an, China, in 2005 and 2011, respectively.

He was a Visiting Student with the University of Sydney, Camperdown, NSW, Australia, from October 2007 to October 2008. He is currently a Tenured Professor with the School of Electronics and Information, Northwestern Polytechnical University. His research interests include hyperspectral remote-sensing image processing and applications, intelligent signal and information acquisition and processing, video processing, and pattern recognition.

Dr. Mei received the First Prize of Natural Science Award of Shaanxi Province in 2022, the Excellent Doctoral Dissertation Award of Shaanxi Province in 2014, the First Place Award certificate for both "Hyperspectral Object Tracking Challenge" and "Cross-City Multimodal Semantic Segmentation Challenge" in WHISPERS 2023. He is serving as an Associate Editor for IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING and IEEE JOURNAL OF SELECTED TOPICS IN APPLIED EARTH OBSERVATIONS AND REMOTE SENSING, a Guest Editor for *Remote Sensing*, and the reviewer for more than 30 international famous academic journals.



Zonghao Han received the B.S. degree in communication engineering from Ningxia University, Yinchuan, China, in 2020. He is currently pursuing the Ph.D. degree in information and communication engineering with the School of Electronics and Information, Northwestern Polytechnical University, Xi'an, China.

His major research interests include hyperspectral remote sensing image processing, image-to-image translation, and deep learning.



Mingyang Ma (Student Member, IEEE) received the B.S. degree in communication engineering and the Ph.D. degree in communication and information systems from Northwestern Polytechnical University, Xi'an, China, in 2015 and 2021, respectively.

He is currently an Assistant Professor with the School of Information and Communications Engineering, Xi'an Jiaotong University, Xi'an. His main research interests include image processing and video summarization.



Fulin Xu received the B.S. degree from Northwestern Polytechnical University, Xi'an, China, in 2021, where he is currently pursuing the master's degree in signal and information processing.

His research interests include hyperspectral remote sensing image processing and deep learning.



Xingang Li received the B.S. degree from Dalian Maritime University, Dalian, China, in 2019, and the M.S. degree from Northwestern Polytechnical University, Xi'an, China, in 2022.

His research interests include hyperspectral remote sensing image processing and deep learning.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:37:13 UTC from IEEE Xplore. Restrictions apply.